

Tìm kiếm ảnh ký tự Hán-Nôm tương đồng bằng học tương phản trên cặp dương sinh từ font

Đề án môn học

Mã nguồn và toàn bộ lệnh tái lập: <https://github.com/vo-hoang-kh4ng/comparing-character-glyph-images-Sino-N-m>

Trọng số: <https://huggingface.co/vohoangkh4ng/chinese-clip-ft-sinonom>

Abstract—Báo cáo trình bày một hệ thống tìm kiếm ảnh ký tự Hán-Nôm tương đồng trên corpus 26.044 glyph, và quá trình cải thiện nó qua ba bước đo được. Thứ nhất, thay bộ trích đặc trưng ResNet18 của bản gốc bằng tháp ảnh Chinese-CLIP, nâng radical@k từ 0,2316 lên 0,5361. Thứ hai, xây dựng phép đánh giá trên ảnh scan thật thay vì chỉ dựa vào chỉ số proxy: 59 ảnh viết tay có nhãn giải mã từ tên file và được người đọc kiểm lại; trên tập này mô hình tốt nhất chỉ đạt $\text{hit@1} = 0,0847$, thấp hơn hẳn mức mà các chỉ số proxy gợi ý. Thứ ba, fine-tune bằng học tương phản có giám sát trên 141.981 ảnh render đa font với nhãn là mã Unicode, đưa hit@1 trên chính 59 ảnh scan đó lên 0,5932 — hai khoảng tin cậy 95% rời hẳn nhau. Trên bài toán xếp hạng trong nhóm ký tự cùng âm Quốc Ngữ, độ chính xác top-1 tăng từ 0,4746 lên 0,8136. Các lớp Unicode bị giữ hoàn toàn ngoài tập huấn luyện đạt kết quả bằng đúng các lớp đã huấn luyện (0,9939 so với 0,9954), nên phần cải thiện không đến từ ghi nhớ. Chúng tôi cũng khảo sát một tầng xếp hạng lại bằng siêu dữ liệu ngôn ngữ học và báo cáo nó như một *kết quả âm*: siêu dữ liệu có trần cao (oracle cho +17,5 điểm) nhưng suy ra siêu dữ liệu của ảnh truy vấn mới là chệch. Mọi kết quả âm — kể cả một lần huấn luyện bị sụp đổ biểu diễn — được giữ lại vì nguyên nhân của chúng có tính lặp lại.

Index Terms—truy hồi ảnh, chữ Nôm, học tương phản, SupCon, Chinese-CLIP, xếp hạng lại, kết quả âm

I. GIỚI THIỆU

Cho một ảnh glyph Hán-Nôm, bài toán là tìm những ký tự trong corpus có hình dạng gần nó nhất. Hệ thống được chia làm hai phần ứng với hai tình huống sử dụng khác nhau. **Phần 1** tìm trên toàn bộ 26.044 ứng viên khi chưa biết ký tự là gì — tình huống khó, dùng khi số hoá một trang chữ chưa được phiên âm. **Phần 2** xếp hạng trong nhóm ký tự cùng âm Quốc Ngữ khi đã biết chữ đó đọc là gì, trung vị 7 ứng viên — tình huống khi hiệu đính một bản phiên âm đã có.

Mục II đặt đề án vào bối cảnh các công trình đã có. Đóng góp của báo cáo, theo thứ tự giá trị:

- 1) Một **phép đánh giá trên dữ liệu thật** thay cho chỉ số proxy: 59 ảnh scan viết tay có nhãn, kèm thang tin cậy do người gán độc lập với mọi điểm số của mô hình (mục V).
- 2) **Fine-tune** (mục IV-C) nâng hit@1 trên scan thật từ 0,0847 lên 0,5932, kèm phép kiểm chứng minh kết quả không phải do ghi nhớ (mục VI-C, VI-D).
- 3) Một **quy trình sinh dữ liệu huấn luyện** từ corpus vốn chỉ có một ảnh mỗi ký tự, bằng render đa font cộng mô hình suy giảm ảnh hiệu chỉnh theo số đo của scan thật (mục IV-B).
- 4) **Năm kết quả âm** được ghi lại đầy đủ, gồm phân tích trần của chỉ số radical@k , một lần sụp đổ biểu diễn khi

huấn luyện, và một tầng xếp hạng lại không đạt kỳ vọng (mục VII, VIII).

II. CÔNG TRÌNH LIÊN QUAN

A. Biểu diễn ảnh dùng chung

Truy hồi ảnh hiện đại hầu như luôn bắt đầu từ một backbone đã pretrain rồi mới tính đến việc huấn luyện riêng. CLIP [9] cho thấy một tháp ảnh học từ cặp ảnh–văn bản quy mô web tạo ra không gian đặc trưng dùng lại được cho nhiều tác vụ mà không cần nhãn của tác vụ đó; Chinese-CLIP [1] lặp lại quy trình ấy trên ngữ liệu tiếng Trung, nên tháp ảnh của nó đã nhìn thấy chữ Hán trong lúc pretrain — lý do chúng tôi chọn nó thay vì CLIP gốc. DINOv2 [2] đại diện cho hướng ngược lại, học hoàn toàn tự giám sát không cần văn bản. Cả hai đều dùng kiến trúc ViT [10]. Baseline của bản gốc là ResNet18 [3] pretrain ImageNet; mục VII cho thấy khoảng cách giữa nó và Chinese-CLIP không nằm ở chi tiết kiến trúc mà mọi người hay đoán.

B. Học metric và học tương phản

Nhánh học metric có giám sát cổ điển dùng hàm mất mát bộ ba [11], và điểm mấu chốt trong thực hành hoá ra không phải công thức mà là *cách lấy mẫu batch*: Hermans và cộng sự [12] chỉ ra rằng batch phải được dựng theo lược đồ $P \times K$ ảnh thì trong batch mới tồn tại cặp dương để so. Đây chính là ràng buộc quyết định ở mục IV-C, và bỏ qua nó là nguyên nhân của lần huấn luyện hỏng ở mục VII-C.

Nhánh tự giám sát dùng InfoNCE [13] với cặp dương sinh bằng tăng cường ảnh, như trong SimCLR [14] và MoCo [15]. SupCon [4] nối hai nhánh lại: giữ dạng InfoNCE nhưng lấy *mọi* ảnh cùng nhãn trong batch làm dương. Với bài toán của chúng tôi đây là dạng tự nhiên nhất — nhãn (mã Unicode) thì có sẵn và miễn phí, còn số lớp thì quá lớn để đặt một lớp phân loại lên trên.

Đối lập lại là nhánh softmax có biên góc, tiêu biểu là ArcFace [5], vốn thống trị nhận dạng khuôn mặt. Điểm khác biệt thường bị bỏ qua: ArcFace cần một ma trận trọng tâm lớp học được cỡ $C \times d$. Ở quy mô nhận dạng mặt, mỗi lớp có hàng chục tới hàng trăm ảnh nên ma trận ấy được ước lượng tốt; ở quy mô của chúng tôi ($C = 23.440$, trung bình 6 ảnh mỗi lớp) thì không. Mục VII-C đo hệ quả. Wang và Isola [16] cung cấp đúng công cụ để chẩn đoán chuyện này: một biểu diễn hỏng vì mất tính *uniformity* trên siêu cầu, và đại lượng đó đo được dễ dàng bằng cosine trung bình giữa các cặp ngẫu

Bảng I
CÁC TẬP DỮ LIỆU. BA TẬP ĐẦU LÀ ĐẦU VÀO CHO TRƯỚC; HAI TẬP SAU DO NHÓM TẠO HOẶC GÁN NHÂN.

Tập	Kích thước	Vai trò
images/	26.044 ảnh	Corpus tra cứu. Mỗi ký tự đúng một ảnh.
final_... QuocNgu...	31.208 dòng —	Bộ thủ, số nét, phân rã IDS. Ảnh xạ âm Quốc Ngữ ↔ ký tự.
test_images/ rendered/	59 ảnh scan 131.604 ảnh	Tập đánh giá thật (viết tay). Sinh ra để huấn luyện, qua 7 font.

nhiên — chỉ số chúng tôi thêm vào vòng đánh giá sau khi mất một lần huấn luyện vì không có nó.

C. Dữ liệu tổng hợp bằng render

Khi nhân thật khan hiếm, render dữ liệu bằng font là một chiến lược đã được kiểm chứng trong nhận dạng chữ: Jaderberg và cộng sự [17] huấn luyện bộ nhận dạng chữ cảnh vật hoàn toàn trên ảnh tổng hợp, và Gupta và cộng sự [18] mở rộng sang định vị chữ. Cả hai đều nhấn mạnh rằng ảnh render sạch *một mình* không đủ; phải có một mô hình suy giảm đưa chúng lại gần phân phối thật. Trong nhận dạng chữ viết tay, biến dạng đàn hồi [19] là ví dụ kinh điển cho ý tưởng ấy. Đóng góp của chúng tôi ở mục IV-B không phải bản thân ý tưởng mà là việc *hiệu chỉnh* hàm suy giảm theo số đo lấy trực tiếp từ ảnh scan mục tiêu, thay vì chọn tham số theo cảm tính, và việc báo cáo khoảng cách còn lại sau khi hiệu chỉnh.

D. Chữ Hán-Nôm

Chữ Nôm dùng lại kho ký tự Hán cộng thêm các ký tự tự tạo của người Việt, phần lớn nằm trong các khối CJK Unified Ideographs Extension của Unicode — trong đó có cả các mặt phẳng ngoài mặt phẳng cơ bản, điều ràng buộc cả lựa chọn font lẫn cách xử lý chuỗi trong toàn bộ mã nguồn. Tri thức ngôn ngữ học thường dùng để hỗ trợ tra cứu là bộ thủ, số nét và phân rã IDS (*Ideographic Description Sequence*) — đúng ba trường có trong bảng tra của đồ án và là đầu vào cho tầng xếp hạng lại ở mục VIII. Điểm chúng tôi muốn nhấn: các trường này rất hấp dẫn để dùng làm *chỉ số đánh giá* vì chúng có sẵn, nhưng mục VII cho thấy chúng là nhân yếu, và mục IV-C giải thích vì sao huấn luyện trên chúng rồi báo cáo chính chúng là một vòng lặp khép kín.

III. DỮ LIỆU

Bảng I liệt kê các tập dữ liệu. Đặc điểm quyết định toàn bộ thiết kế: **corpus chỉ có một ảnh cho mỗi ký tự**. Học metric cần cặp dương — hai ảnh khác nhau của cùng một lớp — nên trong dữ liệu gốc *không tồn tại cặp nào*. Mục IV-B giải quyết đúng chỗ này.

Ảnh trong `test_images/` được đặt tên bằng kiểu gõ Telex (`cofn.jpg` = “còn”), nên giải mã ngược ra chữ Quốc Ngữ rồi tra từ điển sẽ được **tập ký tự đúng** cho ảnh đó. Đây là nhân thật, không phải proxy, và là lý do tập 59 ảnh này có giá trị lớn hơn nhiều so với kích thước của nó.

Bảng II
KHOẢNG CÁCH ĐO ĐƯỢC GIỮA SCAN THẬT VÀ ẢNH RENDER. HÀM SUY GIẢM MÔ PHỎNG ĐÚNG SÁU TRỤC NÀY.

Chỉ số	Scan	Render	Tỉ lệ	Ý nghĩa
edge	15,6	102,3	0,15×	scan mờ hơn nhiều
contrast	91,6	232,8	0,39×	mức xám nhạt
thick	0,051	0,028	1,82×	nét scan dày gấp đôi
fill	0,349	0,206	1,69×	hệ quả của nét dày
noise	10,1	2,7	3,77×	hạt nhiễu giấy
bg	254,0	255,0	≈1×	nền giấy hơi ngà

IV. PHƯƠNG PHÁP

A. Trích đặc trưng và tìm kiếm

Mọi backend đưa ảnh về một vector đã chuẩn hoá L2 nên tích vô hướng chính là cosine. Tìm kiếm dùng Faiss IndexFlatIP [6] — tìm chính xác, không xấp xỉ. Bản gốc dùng ResNet18 [3] pretrain ImageNet với lớp `conv1` thay bằng lớp 1 kênh khởi tạo ngẫu nhiên; chúng tôi giữ bản này nguyên vẹn từng bit làm mốc “trước”, và thêm năm backend để so, gồm ba cỡ Chinese-CLIP [1] và DINOv2 [2].

B. Sinh cặp dương bằng render đa font

Vì mỗi ký tự chỉ có một ảnh, chúng tôi render lại từng mã Unicode qua bảy font Hán-Nôm khác nhau: cùng một lớp, khác kiểu chữ. Độ phủ được kiểm bằng bảng `cmap` của từng font chữ không phỏng đoán; hợp nhất lại phủ 26.033/26.044 ký tự. **Ký tự không có trong cmap bị bỏ qua**, vì nếu render bừa thì font trả về ô tofu — một ô vuông rỗng giống hệt nhau ở mọi ký tự — đưa vào huấn luyện sẽ đầu độc cặp dương.

Nhưng tập render sạch *quá dẽ*: ảnh render tự truy hồi đúng chữ ở `hit@1` = 0,7018 trong khi scan thật chỉ đạt 0,0847, chênh gần chín lần. Nghĩa là phần lớn khoảng cách tới thực tế không nằm ở font. Chúng tôi đo sáu chỉ số ảnh trên hai tập để tìm chỗ lệch (Bảng II) rồi xây hàm `degrade()` mô phỏng đúng sáu trục đó theo thứ tự vật lý của một trang scan. Mọi phép biến đổi đều là nhiễu ảnh, *không đụng tới cấu trúc bộ thủ*, nên nhân vẫn đúng. Kết quả: `hit@1` của tập render tụt từ 0,7018 xuống 0,1986 — chỉ còn dẽ hơn scan thật 2,8 lần thay vì 9 lần.

Phần 2,8 lần còn lại là khác biệt **cấu trúc** (bút lông, lối hành thư: nét nối liền, tỉ lệ bộ phận xê dịch, có nét bị lược) và không thể sinh ra từ một ảnh in bằng xử lý ảnh.

C. Fine-tune

Mục tiêu học là **bất biến kiểu chữ**: cùng một mã Unicode qua font khác nhau phải cho cùng một vector. Nhân duy nhất là mã Unicode — cố ý *không* dùng `RADICAL` hay `STROKE_NUM`, vì huấn luyện trên một nhân ngôn ngữ học rồi báo cáo chính chỉ số của nhân đó là đo lại tập huấn luyện chứ không phải đánh giá.

Công thức hàm mất mát và lược đồ lấy mẫu được viết đầy đủ ở mục IV-D. Hai lựa chọn trong Bảng III là **điều kiện sống còn chứ không phải tinh chỉnh**, và cả hai đều đến từ lần thất bại ở mục VII-C:

Bảng III
CẤU HÌNH FINE-TUNE.

Thành phần	Lựa chọn
Backbone	Chinese-CLIP ViT-B/16, giữ nguyên visual_projection (512 chiều)
Hàm mất mát	SupCon [4], nhiệt độ 0,07
Lấy mẫu dữ liệu	P×K: 24 lớp × 2 view mỗi batch 141.981 ảnh / 23.440 lớp
Suy giảm	degrade() chạy trực tuyến trong DataLoader, $p = 0,6$
Tối ưu	AdamW, lr 10^{-5} , warmup 300, cosine, bf16
Chi phí	3,7 GB VRAM , ~4 phút/epoch, 6 epoch

- **Lấy mẫu P×K.** Một batch 48 ảnh rút ngẫu nhiên từ 23.440 lớp chỉ chứa khoảng 0,05 cặp dương theo kỳ vọng — gần như mọi batch sẽ không có gì để kéo lại gần nhau, và hàm mất mát trở thành hằng số.
- **SupCon thay vì softmax có biên góc.** SupCon không có tham số học được nào trong hàm mất mát, nên không có một head khởi tạo ngẫu nhiên bơm nhiễu ngược về backbone.

Suy giảm ảnh chạy *trực tuyến lúc nạp dữ liệu* chứ không render sẵn ra đĩa: bộ sinh số ngẫu nhiên gieo theo bộ ba (seed, epoch, chỉ số ảnh) nên mỗi epoch mỗi ảnh có một biến thể khác, vẫn tái lập được, và không tốn thêm dung lượng nào.

D. Hàm mất mát và lược đồ lấy mẫu, viết đầy đủ

Gọi B là batch, $z_i = f_\theta(x_i)/\|f_\theta(x_i)\|_2$ là embedding đã chuẩn hoá, y_i là mã Unicode của ảnh x_i , và $\tau = 0,07$. Đặt $P(i) = \{p \in B : y_p = y_i, p \neq i\}$ là tập dương của neo i . Hàm mất mát SupCon [4] là

$$\mathcal{L} = \frac{1}{|A|} \sum_{i \in A} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i^\top z_p / \tau)}{\sum_{a \in B \setminus \{i\}} \exp(z_i^\top z_a / \tau)}, \quad (1)$$

trong đó $A = \{i \in B : |P(i)| > 0\}$. Ba chi tiết cài đặt không đọc ra được từ (1) nhưng quyết định việc nó có chạy hay không:

- 1) **Mẫu số bỏ chính neo**, nên đường chéo của ma trận tương đồng phải bị che. Cách che chuẩn là gán $-\infty$ trước khi lấy logsumexp .
- 2) $-\infty \cdot 0 = \text{NaN}$ theo **IEEE 754**. Sau logsumexp , đường chéo vẫn còn $-\infty$; nhân nó với mặt nạ dương (giá trị 0 tại đường chéo) cho NaN, và toàn bộ gradient thành NaN ngay bước đầu tiên. Phải *gán lại đường chéo về 0* sau logsumexp và trước phép nhân mặt nạ. Đây là lỗi chúng tôi thực sự gặp.
- 3) **Neo không có dương phải bị loại khỏi trung bình**, không phải cho bằng 0: gộp chúng vào làm hàm mất mát bị chia nhỏ tùy theo có bao nhiêu lớp lẻ trong batch.

Giá trị của (1) khi biểu diễn sụp đổ hoàn toàn (mọi z_i trùng nhau) là $\ln(PK - 1) = \ln 47 \approx 3,85$; con số này là *dấu hiệu chẩn đoán* chứ không phải một mức mất mát ngẫu nhiên, và

biết nó cho phép phát hiện sụp đổ chỉ bằng cách nhìn hàm mất mát.

Lược đồ lấy mẫu được viết thành một Sampler riêng thay vì trộn ngẫu nhiên:

Thuật toán 1: PKSampler (nhân, batch=48, K=2)
1 $P \leftarrow \max(2, \text{batch} // K)$ # 24 lớp mỗi batch
2 bỏ mọi lớp có $< K$ ảnh # không dùng nổi cặp dương
3 lặp mỗi epoch:
4 hoán vị ngẫu nhiên danh sách lớp
5 với mỗi khối P lớp liên tiếp:
6 với mỗi lớp trong khối:
7 lấy K ảnh, không lặp trong epoch
8 trả về P·K chỉ số làm một batch

Vì sao bước 1–2 là bắt buộc chứ không phải tinh chỉnh: với $C = 23.440$ lớp và batch ngẫu nhiên cỡ $B = 48$, số cặp dương kỳ vọng trong batch là

$$\mathbb{E}[\text{\#cặp dương}] = \binom{B}{2} \Pr[y_i = y_j] \approx 1128 \cdot \frac{1}{23.440} \approx 0,05. \quad (2)$$

Tức là **khoảng 95% số batch hoàn toàn không có gì để kéo lại gần nhau**, và với những batch đó (1) không có số hạng nào — hàm mất mát trở thành hằng số và gradient bằng không. Không có công thức mất mát nào cứu được chuyện này; nó là lỗi ở tầng lấy mẫu.

V. PHƯƠNG PHÁP ĐÁNH GIÁ

Phần lớn công sức của đồ án nằm ở chỗ này, vì một chỉ số sai làm mọi kết luận phía sau vô nghĩa. Ba tầng đánh giá, xếp theo độ tin cậy tăng dần: *chỉ số proxy* trên toàn corpus (radical@k, stroke_mae@k, ids_jaccard@k) — rẻ nhưng là nhân yếu; *nhân thật từ tên file* trên 59 ảnh scan, với mức ngẫu nhiên khoảng $4,7 \cdot 10^{-4}$; và *nhân do người gán* cho Phần 2, đã gán đủ 59/59 ảnh.

Cột *confidence* trong bảng nhân là **đánh giá của người đọc ảnh, không suy ra từ bất kỳ điểm số nào của mô hình**. Điều này là cố ý: phép kiểm độ nhạy dùng chính cột này để lọc ra tập nhân chắc chắn rồi chấm lại mô hình trên đó, nên nếu *confidence* lấy từ điểm similarity thì phép kiểm ấy thành vòng tròn — lấy mô hình chấm mô hình. Thang gồm ba mức *high* (nét quyết định đọc được), *med* (còn ít nhất một biến thể gần trùng chỉ khác ở chi tiết đã mờ) và *low* (lựa chọn dựa vào văn cảnh nhiều hơn hình), phân bố cuối cùng 53/4/2.

Các ảnh scan là những dòng mở đầu *Truyện Kiều*, nên nhân được gán bằng so hình trước rồi đối chiếu với chính tả Nôm chuẩn của tác phẩm. **Khi hình và văn cảnh mâu thuẫn thì hình thắng**, và *confidence* bị hạ xuống *med* — bản scan viết “qua” là 𠂔 chứ không phải 過, “năm” là 𠂔 chứ không phải 年.

A. Định nghĩa chỉ số

Để tránh mơ hồ, các chỉ số được viết ra đầy đủ. Với N truy vấn, gọi r_q là thứ hạng (bắt đầu từ 1) của ứng viên đúng đầu tiên cho truy vấn q ; nếu không có ứng viên đúng nào trong danh sách trả về thì $1/r_q := 0$.

Bảng IV

TOÀN BỘ CORPUS 26.044 GLYPH, TOP- $K = 20$. ĐẬM LÀ TỐT NHẤT MỖI CỘT.

Backend	radical@k ↑	stroke ↓	ids @k ↑	img/s
resnet18 (gốc)	0,2316	2,8942	0,1031	3950
resnet18-gray	0,2202	2,5658	0,1020	3937
chinese-clip B/16	0,4843	2,6296	0,1997	358
chinese-clip-large	0,5361	2,6992	0,2113	354
chinese-clip-huge	0,5147	2,6547	0,2144	283
dinov2	0,3081	2,3397	0,1251	306

$$\text{hit}@k = \frac{1}{N} \sum_{q=1}^N \mathbf{1}[r_q \leq k], \quad \text{MRR} = \frac{1}{N} \sum_{q=1}^N \frac{1}{r_q}. \quad (3)$$

$\text{radical}@k$ là chỉ số *proxy*: tỉ lệ trong k ứng viên đầu có cùng bộ thủ với truy vấn, lấy trung bình trên mọi truy vấn. Nó không cần nhãn thật, đó vừa là ưu điểm (chạy được trên cả 26.044 ký tự) vừa là nhược điểm — mục VII đo trần của chính nó.

Khoảng tin cậy dùng công thức Wilson [8] chứ không dùng xấp xỉ chuẩn (Wald). Với $\hat{p}$ là tỉ lệ quan sát, n là cỡ mẫu và $z = 1,96$:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}. \quad (4)$$

Lý do: ở $n = 59$ và các tỉ lệ gần 0 mà chúng tôi thực sự đo được ở cấu hình zero-shot ($\hat{p} = 0,0847$), khoảng Wald tràn qua 0 và cho cận dưới âm — một khoảng tin cậy vô nghĩa cho một tỉ lệ. Wilson không bao giờ ra ngoài $[0, 1]$.

Mọi tỉ lệ đều kèm khoảng tin cậy nhị thức 95% [8]. Với $n = 59$ và $p \approx 0,5$, khoảng tin cậy rộng khoảng ± 13 điểm, nên bất kỳ cải thiện nào dưới ~ 15 điểm đều không chứng minh được. Đây là giới hạn cứng của tập test hiện có, và báo cáo tuân thủ nó một cách nhất quán: trong toàn bộ đồ án chỉ có đúng một so sánh vượt qua được ngưỡng này.

VI. KẾT QUẢ

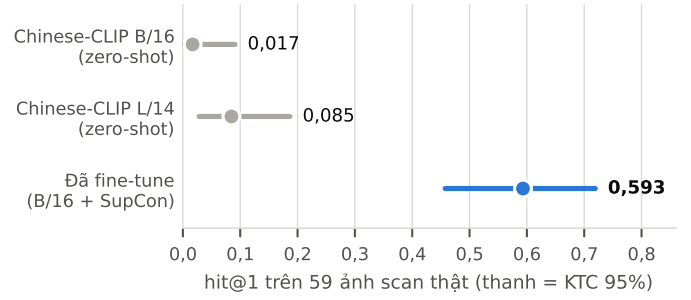
A. So sánh backend trên toàn corpus

Bảng IV cho kết quả. Chinese-CLIP thắng với biên độ lớn: $\text{radical}@k$ gấp **2,31 lần** bản gốc và ids_jaccard gấp 2,05 lần, mà gần như không tốn thêm thời gian (354 so với 358 ảnh/giây). DINOv2 thua ở bộ thủ nhưng *thắng ở số nét*, và chỉ trùng 12,9% kết quả top- k với họ CLIP, nên nó không đơn thuần là kém hơn mà mạnh ở một trục khác.

B. Trần của chính chỉ số proxy

Một câu hỏi được đặt ra: $\text{radical}@k \approx 0,5$ có phải đã bão hòa? Giả thuyết “tập top- k thiếu ứng viên đúng” bị bác bỏ: trần lý thuyết của $\text{radical}@20$ là 0,9862, tức chỉ 3% query bị chạm trần; và $\text{radical}@1$ cũng chỉ đạt 0,6331, nên chỗ nghẽn nằm ngay ở lảng giềng gần nhất chứ không ở đuôi danh sách.

Nhưng phép kiểm làm lộ ra một điều quan trọng hơn: **RADICAL không phải nhãn thị giác thuần túy**. Bộ thủ chỉ



Hình 1. Kết quả chính. $\text{hit}@1$ khi tìm trên toàn bộ 26.044 ký tự, đo trên 59 ảnh scan thật. Thanh ngang là khoảng tin cậy nhị thức 95%. **Hai khoảng của bản zero-shot và bản fine-tune rời hẳn nhau** — đây là so sánh duy nhất trong đồ án mà $n = 59$ đỡ nổi một kết luận.

Bảng V

PHẦN 2 — XẾP HẠNG TRONG NHÓM CÙNG ÂM QUỐC NGỮ, $n = 59$.

Phương pháp	Đúng top-1	MRR	KTC 95%
Chọn ngẫu nhiên trong nhóm	0,1987	—	—
histogram (bản gốc)	0,4407	0,6070	[0,312; 0,576]
embedding zero-shot	0,4746	0,6566	[0,343; 0,609]
embedding fine-tune	0,8136	0,8983	[0,691; 0,903]

thực sự xuất hiện trong hình chữ ở 56,7% số ký tự; phần còn lại mang bộ thủ vì lý do từ nguyên hoặc quy ước tra cứu, không phải vì trông giống. Một oracle *biết trước* phân rã IDS cũng chỉ đạt $\text{radical}@20 = 0,7338$. So với trần thực tế đó, *chinese-clip-large* (0,5361) đang ở **73% quãng đường** và 24,4 lần mức ngẫu nhiên. Kết luận: chưa bão hòa, nhưng cũng không nên tiếp tục tối ưu cho một chỉ số mà bản thân nó chỉ đúng ở hai phần ba.

C. Kết quả trên ảnh scan thật

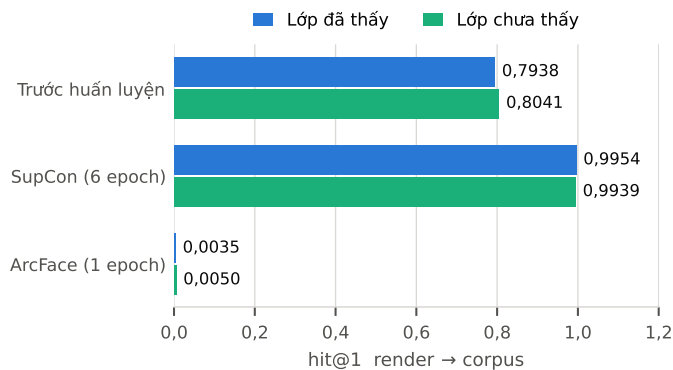
Đây là phép đo duy nhất không phải proxy, và là con số nên dùng để đánh giá đồ án. Không một pixel nào của `test_images/` đi vào huấn luyện; điều này được bảo đảm bằng một `assert` chạy trước mỗi lần huấn luyện chứ không bằng quy ước.

Hình 1 cho thấy khoảng cách. Bản fine-tune đạt $\text{hit}@1 = 0,5932$ [0,457; 0,719] so với 0,0847 [0,028; 0,187] của *chinese-clip-large* zero-shot — 35/59 so với 5/59, hơn mức ngẫu nhiên 1.250 lần. $\text{hit}@10$ là 0,7966 và MRR là 0,6691.

Trước fine-tune, kết quả thấp **không phải vì mô hình kém mà vì lệch phân phối**. Triệu chứng là hiệu ứng hub: ký tự 攢 được trả về ở vị trí một cho sáu query khác nhau. Query là chữ viết tay lối hành thư, corpus là khẩu thư in bằng font. Chúng tôi đã thử chuẩn hoá hình học (cắt sát nét mực, đệm vào khung vuông) và kết quả nằm trong khoảng nhiều — phần lệch hình học có thật nhưng không phải nguyên nhân chính.

Bảng V cho Phần 2. Phép thử độ nhạy trên 53 nhãn *high* cho 0,8491 so với histogram 0,4717, nên kết luận không phụ thuộc vào sáu nhãn *med/low* còn lại.

Bảng này **sửa lại một kết luận trước đó của chính nhóm**. Với mô hình zero-shot, khoảng cách giữa embedding và histogram chỉ là *hai ảnh trên 59* — nằm gọn trong nhiễu — nên khuyến nghị dùng embedding khi ấy phải dựa vào chất



Hình 2. Phép kiểm tổng quát hoá. 2.604 mã Unicode bị gỡ *toàn bộ view* khỏi tập huấn luyện — chia theo *lớp* chứ không theo ảnh. Sau SupCon, lớp chưa thấy đạt bằng lớp đã thấy. ArcFace sụp ở *cả hai*, dấu hiệu của sụp đổ biểu diễn chứ không phải overfit.

lượng điểm số chữ không đảm bảo độ chính xác. Sau fine-tune, khoảng cách là 22 ảnh và hai khoảng tin cậy rời nhau: độ chính xác đã đủ để tự đứng làm lý do.

D. Có phải mô hình chỉ ghi nhớ 23.440 danh tính?

Đây là phản biện hiển nhiên nhất với một kết quả tăng mạnh như vậy, nên chúng tôi thiết kế phép kiểm cho nó ngay từ đầu. Chia theo lớp chứ không theo ảnh, vì chia theo ảnh thì lớp nào cũng có mặt ở cả hai bên và phép đo mất sạch ý nghĩa. Hai mẫu cùng cỡ 2.604 và cùng cách chọn ngẫu nhiên — chọn theo thứ tự mã Unicode sẽ thiên về khối CJK cơ bản trong khi held-out rải đều cả Ext-B.

Hình 2 cho kết quả: 0,9954 so với **0,9939**. Bằng nhau. Điều này khớp với cơ chế: SupCon không có tâm lớp học được nên không có gì để ghi nhớ — thứ nó học là *phép so hai ảnh*, và phép đó chuyển sang ký tự chưa từng gặp nguyên vẹn.

E. Quan sát định tính

Hình 3 cho thấy mô hình *làm gì* chứ không chỉ ghi được bao nhiêu điểm. Đáng chú ý ở truy vấn “chữ”: bản fine-tune trả về hai biến thể đúng ở hai vị trí đầu, đều mang thành phần 字, trong khi bản zero-shot trả về những chữ chỉ giống ở mặt độ nét.

VII. KẾT QUẢ ÂM

Bốn kết quả dưới đây đều là thất bại hoặc bác bỏ, và đều được giữ lại vì chúng loại trừ được những hướng mà người đọc báo cáo ngày rất dễ thử lại.

A. Giả thuyết “lỗi nằm ở conv1” — sai

Bản gốc thay conv1 bằng lớp 1 kênh khởi tạo ngẫu nhiên, tức vứt bỏ bộ lọc tầng đầu đã pretrain. Giả thuyết tự nhiên là chỉ cần vá dòng đó. Chúng tôi dựng resnet18-gray gieo conv1 bằng tổng các bộ lọc RGB pretrain — kết quả radical@k **giảm nhẹ** (0,2202 so với 0,2316). Sửa một dòng không cứu được kiến trúc; đây chính là bằng chứng biện minh cho việc đổi hẳn sang ViT thay vì vá baseline.



Hình 3. Năm lảng giềng đầu cho ba ảnh scan thật, trước và sau fine-tune. Khung xanh = ký tự đúng. Truy vấn được chọn theo quy tắc cố định — những ảnh mà bản fine-tune đúng ở hạng 1 còn zero-shot sai ở hạng 1, lấy theo thứ tự tên file — chữ không bởi tìm ví dụ đẹp. Bản zero-shot trả về các ký tự không liên quan về cấu trúc; bản fine-tune trả về đúng chữ và các biến thể cùng bộ.

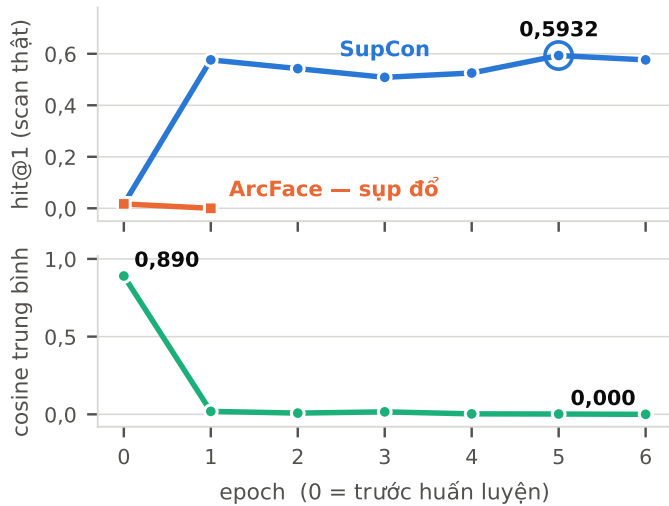
B. Scale có lợi, rồi đảo chiều

base → large là +10,7% tương đối trên radical@k, nhưng large → huge là −4,0% trong khi tốn thêm 33% số chiều và 20% thời gian. Đường cong đã quay đầu. Đây là chữ ký của “thêm dung lượng cho một mục tiêu huấn luyện sai domain” — và cũng chính là lý do khiến fine-tune trở thành hướng đáng làm nhất, thay vì đổi sang checkpoint lớn hơn.

C. ArcFace gây sụp đổ biểu diễn

Thiết kế fine-tune đầu tiên dùng ArcFace [5] trên 23.440 lớp (scale 32, biên 0,30). Sau một epoch, hit@1 trên scan thật về 0 và truy hồi render→corpus rơi từ 0,79 xuống 0,0035 trên lớp đã thấy và 0,80 xuống 0,0050 trên lớp chưa thấy (Hình 2, Hình 4).

Sụp ở *cả hai* phía nên đây không phải overfit mà là **sụp đổ biểu diễn**: mọi ảnh dồn về một vector. Nguyên nhân nằm ở hình dạng dữ liệu chứ không phải siêu tham số — 23.440 lớp mà mỗi lớp chỉ khoảng 6 ảnh. Head ArcFace khởi tạo ngẫu nhiên với 23.440 tâm không bao giờ kịp tổ chức, nên thứ nó truyền ngược về backbone gần như là nhiễu. Và vì Adam chuẩn hoá theo độ lớn gradient, “tốc độ học nhỏ” 10^{-5} vẫn dịch chuyển trọng số đủ để phá cấu trúc pretrain trong khoảng 3.000 bước; cắt chuẩn gradient không cứu được, vì nhiễu bị cắt chuẩn vẫn là nhiễu.



Hình 4. Động lực huấn luyện. *Trên*: hit@1 trên scan thật qua từng epoch; ArcFace mất toàn bộ khả năng truy hồi sau một epoch. *Dưới*: cosine trung bình giữa 2.000 vector corpus ngẫu nhiên — máy dò sụp đổ. Con số 0,890 lúc zero-shot tự nó giải thích vì sao khả năng phân biệt trước đây yếu; sau huấn luyện không gian giãn ra gần trực giao. Hai đại lượng khác thang nên vẽ ở hai bảng riêng, không dùng hai trục y chồng nhau.



Hình 5. Bộ bên trái của một ảnh scan, phóng to. Nét dọc *dùng* ở vạch ngang nên đây là 招 chứ không phải 𠂔 (móc kéo xuống dưới vạch). Sửa nhân này làm khoảng cách giữa hai phương pháp co từ bốn ảnh xuống hai — tức đi ngược lợi ích của kết luận nhóm đang muốn chứng minh.

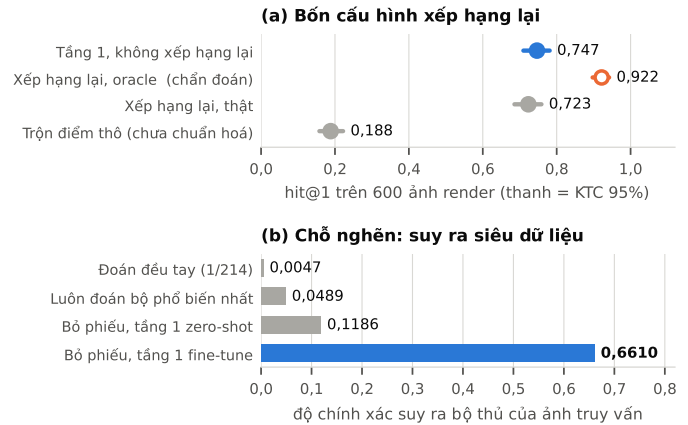
Ba công cụ chẩn đoán được thêm vào sau lần này, và nên có trong bất kỳ thí nghiệm tương tự nào: (a) đánh giá *giữa epoch* để sự cố lộ ra sau một phút thay vì sau một giờ; (b) cột đo **cosine trung bình** ở Hình 4 — nó chạy về 1,0 khi sụp đổ, và nếu không có nó thì “học hỏng” với “sụp đổ” trông giống hệt nhau trên hit@1 trong khi hai thứ đó phải sửa theo hai cách khác nhau; (c) biết trước chữ ký sụp đổ của SupCon là hàm mất mát đúng đúng ở $\ln(P \cdot K - 1)$.

D. Một nhân sai đã tự làm giảm kết quả của chính nhóm

Khi soát lại 11 dòng *med/low*, chúng tôi phát hiện một nhân gán sai (Hình 5). Ghi lại điều này vì nó là thước đo trực tiếp cho việc một bảng số ở $n = 59$ nhảy tới mức nào với đúng một nhân.

VIII. XẾP HẠNG LẠI BẰNG SIÊU DỮ LIỆU

Ba cột siêu dữ liệu ngôn ngữ học (RADICAL, STROKE_NUM, SHAPE_MORPH) mô tả những thuộc tính mà mô hình ảnh không nhìn thấy. Câu hỏi tự nhiên: dùng chúng để **xếp hạng lại** danh



Hình 6. (a) Bốn cấu hình xếp hạng lại trên 600 ảnh render, tầng 1 là chinese-clip-large. Cấu hình oracle vẽ rộng ruột vì nó *cố tình* đọc trộm siêu dữ liệu thật của truy vấn — một chẩn đoán, không phải kết quả. (b) Chỗ nghẽn đã định lượng: độ chính xác suy ra bộ thủ của ảnh truy vấn, so với hai mốc đoán mò.

sách ứng viên mà tầng ảnh trả về thì có tốt hơn không? Câu trả lời là *chưa*, và lý do cụ thể hơn nhiều so với “không hiệu quả”.

A. Vì sao không thể chấm bằng chỉ số cũ

Ba chỉ số proxy được suy ra từ *đúng ba cột* mà tầng xếp hạng lại dùng làm đặc trưng. Chấm nó bằng *radical@k* là vòng tròn: điểm số sẽ leo về trần 0,978 mà không chứng minh được gì, vì mô hình được đưa sẵn đáp án. Phép đo trung thực phải dùng nhãn **độc lập** với ba cột đó — ở đây là mã Unicode của chính ký tự, trên tập ảnh render ($n = 600$, khoảng tin cậy $\pm 3,5$ điểm).

B. Ba điều rút ra

Từ Hình 6(a), theo thứ tự quan trọng:

- **Chuẩn hoá tín hiệu trước khi trộn là bắt buộc.** Trộn điểm thô làm mất 56 điểm hit@1 ($0,7467 \rightarrow 0,1883$). Lý do: với trọng số mặc định, một ứng viên phải thắng về độ giống ảnh hơn 0,364 cosine mới bù nổi một lần lệch bộ thủ, mà khoảng cách cosine trong top-20 không bao giờ rộng đến thế. Trộn thô vì vậy biến phép xếp hạng thành **sắp xếp từ điển** — bộ thủ trước, rồi số nét — đẩy mô hình ảnh xuống làm tiêu chí phá hoà.
- **Siêu dữ liệu có trần cao.** Cấu hình oracle được +17,5 điểm. Vấn đề không nằm ở ý tưởng.
- **Nhưng trần đó chưa với tới được.** Cấu hình thật đạt 0,7233, tức *hơi thấp* hơn mốc không xếp hạng lại.

C. Chỗ nghẽn, định lượng

Ảnh truy vấn là một bản scan chưa biết là chữ gì nên không tra được siêu dữ liệu của nó; cách thay thế là **bỏ phiếu từ láng giềng**. Hình 6(b) đo trực tiếp bước này: với tầng 1 zero-shot, phép bỏ phiếu chỉ đúng 11,86% — hơn đoán mò 2,4 lần, nhưng **sai 88% số lần**, trong khi bộ thủ chiếm trọng số 0,20 trong điểm tổng. Tầng xếp hạng lại đang được nuôi bằng nhiều.

Nguyên nhân có tính cơ chế: phép bỏ phiếu lấy từ láng giềng của tầng 1, mà tầng 1 zero-shot chỉ đúng 8,47% trên scan thật, nên đa số phiếu là của chữ sai. **Thay tầng 1 bằng mô hình đã fine-tune thì độ chính xác nhảy lên 0,6610 — gấp 5,6 lần.** Mất xích yếu không nằm ở thiết kế của phép suy luận mà ở chất lượng của tầng đứng trước nó.

D. Một cảnh báo về phép đo

Tập ảnh render *không* dùng để đánh giá mô hình đã fine-tune được, vì chính chúng là dữ liệu huấn luyện của nó. Chúng tôi đã thử cứu bằng cách chỉ lấy 2.604 lớp held-out, rồi thử tiếp trên bản đã suy giảm — cả hai đều bảo hoà trên 0,98. Nghĩa là phép đo không vòng tròn ở trên chỉ dùng được cho mô hình zero-shot.

IX. THẢO LUẬN

Bốn điều dưới đây rút ra từ đồ án nhưng không phụ thuộc vào chữ Nôm, nên chúng tôi tách riêng.

A. Chỉ số proxy đo được không có nghĩa là đo đúng

Chỉ số `radical@k` rẻ, chạy trên toàn bộ 26.044 ký tự, và tăng đơn điệu khi đổi backbone — ba tính chất khiến nó trông như một chỉ số tốt. Nhưng cấu hình đứng đầu theo chỉ số ấy chỉ đạt `hit@1` = 0,0847 trên ảnh scan thật. Khoảng cách không nằm ở phương sai mà ở *định nghĩa*: hai ký tự cùng bộ thủ có thể chẳng giống nhau chút nào về hình. Bài học không phải “dùng dùng proxy” — proxy vẫn cần thiết ở giai đoạn sàng lọc — mà là **phải đo trần của proxy trước khi tin vào thứ hạng nó sinh ra**, và 59 ảnh có nhãn thật đã đủ để lật ngược kết luận rút ra từ 26.044 ảnh có nhãn yếu.

B. Lỗi ở tầng lấy mẫu đội lốt lỗi ở tầng hàm mất mát

Khi lần huấn luyện ArcFace sụp đổ, phản xạ đầu tiên là đổi hàm mất mát, hạ tốc độ học, cắt gradient — tất cả đều nhắm vào tầng tối ưu. Không cái nào có tác dụng, vì nguyên nhân thật nằm ở phương trình (2): cấu trúc batch. Dấu hiệu phân biệt là một dấu hiệu chung, không riêng gì bài này: **kết quả giảm trên cả lớp đã thấy lẫn lớp chưa thấy là sụp đổ biểu diễn, không phải quá khớp**. Quá khớp thì tăng ở tập huấn luyện. Một dòng cosine trung bình giữa các cặp ngẫu nhiên [16] trong vòng đánh giá phát hiện chuyện này trong một phút, và chi phí thêm gần bằng không.

C. Một kết quả âm chỉ dùng được khi kèm trần của nó

Tầng xếp hạng lại bằng siêu dữ liệu không cải thiện kết quả. Câu “không hiệu quả” tự nó không nói được gì — có thể tín hiệu vô dụng, có thể cài đặt sai, có thể tham số chưa chỉnh. Phép đo oracle (giả sử biết đúng siêu dữ liệu của truy vấn) tách được ba khả năng đó ra: siêu dữ liệu cho +17,5 điểm nếu biết đúng, còn bước suy ra siêu dữ liệu từ ảnh chỉ đạt 11,86%. Kết luận đổi từ “hướng này hỏng” thành **“hướng này còn dư địa lớn, và chỗ cần sửa là bộ suy luận bộ thủ”** — một kết luận có thể hành động được. Chúng tôi cho rằng mọi kết quả âm nên kèm một phép đo oracle rẻ tiền như vậy.

D. Kỷ luật khoảng tin cậy thay đổi kết luận nào được viết ra

Với $n = 59$, khoảng tin cậy 95% rộng khoảng ± 13 điểm. Áp dụng ngưỡng này một cách nhất quán thì **trong toàn bộ đồ án chỉ đúng một so sánh sống sót**: zero-shot so với fine-tune. Mọi thứ tự hạng giữa các backbone, mọi khác biệt giữa các epoch, mọi biến thể của tầng xếp hạng lại đều nằm trong nhiễu và được báo cáo đúng như vậy. Chi phí là bản báo cáo có ít khẳng định mạnh hơn; đổi lại, những khẳng định còn lại thì đứng vững được.

X. HẠN CHẾ

- $n = 59$ là **toàn bộ dữ liệu scan hiện có**, và là nút thắt lớn nhất. Mọi so sánh trong đồ án trừ mục VI-C đều nằm trong khoảng nhiễu. Cần $n \approx 150$ để khoảng tin cậy xuống ± 8 điểm.
- **Mô hình fine-tune chỉ được đo trên một lần chạy, một seed**. Chênh lệch giữa các epoch (0,51–0,59) đã bằng cỡ nhiễu của phép đo, nên 0,5932 nên đọc là “quanh 0,55”.
- **Backbone là ViT-B/16 chứ không phải large**, vì large không vừa VRAM lúc huấn luyện trên máy dùng chung. Chưa biết fine-tune large sẽ cho kết quả bao nhiêu.
- **Tầng xếp hạng lại chưa được kiểm chứng ở cấu hình tốt nhất**. Hình 6 đo với tầng 1 zero-shot; chưa ai chạy lại đủ bốn cấu hình trên tầng 1 fine-tune.
- Nhân Phần 2 do **so hình** mà ra, không phải do chuyên gia Hán-Nôm đọc. Còn 6/59 dòng ở mức *med/low*.
- Ảnh query là chữ viết tay lỗi hành thư còn corpus là khai thư in — mọi con số đo trên đúng một cặp phân phối này.
- Recall của chỉ mục xấp xỉ [7] chỉ đo ở $N = 26.044$; recall của HNSW thường giảm khi N tăng, nên 99,1% không được suy diễn cho mốc một triệu.

XI. KẾT LUẬN

Mục IX đã tách riêng những điều khái quát ra ngoài bài toán chữ Nôm; phần này chỉ tóm tắt các kết luận về chính hệ thống. Môi trường và chi phí tính toán để tái lập nằm ở Phụ lục B, cấu trúc gói nộp ở Phụ lục C và Bảng VII.

- 1) **Chuyển sang Chinese-CLIP**. Hơn baseline ResNet18 2,31 lần ở `radical@k` với chi phí gần như bằng không. Không scale tiếp lên ViT-H: đường cong đã quay đầu.
- 2) **Fine-tune là đòn bẩy lớn nhất, và đã đo được**. `hit@1` trên scan thật đi từ 0,0847 lên **0,5932**, Phần 2 từ 0,4746 lên **0,8136**, với bằng chứng rằng phần cải thiện không đến từ ghi nhớ.
- 3) **Giữ tìm kiếm chính xác** `IndexFlatIP` cho tới khi corpus vượt khoảng 300.000 ký tự; search chỉ chiếm 6% thời gian ở cỡ hiện tại.
- 4) **Xếp hạng lại bằng siêu dữ liệu chưa dùng được, nhưng biết rõ vì sao**. Oracle cho thấy còn +17,5 điểm dư địa; chỗ nghẽn là bước suy ra bộ thủ của ảnh truy vấn.
- 5) **Việc tiếp theo, theo thứ tự chi phí**. (a) *Hard negative mining*: hàm mất mát hiện rơi về 0,0068 chỉ sau 400 bước vì 24 lớp lấy ngẫu nhiên từ 23.440 thì phân biệt quá dễ. (b) Scan thêm ảnh thật để gỡ nút thắt $n = 59$.

(c) Bổ sung dữ liệu thư pháp bút lông thật — thứ mà render font về nguyên tắc không sinh ra được.

TÀI LIỆU THAM KHẢO

[1] A. Yang và cộng sự, “Chinese CLIP: Contrastive Vision-Language Pretraining in Chinese,” arXiv:2211.01335, 2022.

[2] M. Oquab và cộng sự, “DINOv2: Learning Robust Visual Features without Supervision,” arXiv:2304.07193, 2023.

[3] K. He, X. Zhang, S. Ren và J. Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016.

[4] P. Khosla và cộng sự, “Supervised Contrastive Learning,” NeurIPS, 2020.

[5] J. Deng, J. Guo, N. Xue và S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” CVPR, 2019.

[6] J. Johnson, M. Douze và H. Jegou, “Billion-scale similarity search with GPUs,” IEEE Transactions on Big Data, 2019.

[7] Y. A. Malkov và D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” IEEE TPAMI, 2020.

[8] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” J. Amer. Statist. Assoc., 1927.

[9] A. Radford và cộng sự, “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.

[10] A. Dosovitskiy và cộng sự, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” ICLR, 2021.

[11] F. Schroff, D. Kalenichenko và J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” CVPR, 2015.

[12] A. Hermans, L. Beyer và B. Leibe, “In Defense of the Triplet Loss for Person Re-Identification,” arXiv:1703.07737, 2017.

[13] A. van den Oord, Y. Li và O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv:1807.03748, 2018.

[14] T. Chen, S. Kornblith, M. Norouzi và G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” ICML, 2020.

[15] K. He, H. Fan, Y. Wu, S. Xie và R. Girshick, “Momentum Contrast for Unsupervised Visual Representation Learning,” CVPR, 2020.

[16] T. Wang và P. Isola, “Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere,” ICML, 2020.

[17] M. Jaderberg, K. Simonyan, A. Vedaldi và A. Zisserman, “Synthetic Data and Artificial Neural Networks for Natural Scene Text Recognition,” NIPS Deep Learning Workshop, 2014.

[18] A. Gupta, A. Vedaldi và A. Zisserman, “Synthetic Data for Text Localisation in Natural Images,” CVPR, 2016.

[19] P. Y. Simard, D. Steinkraus và J. C. Platt, “Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis,” ICDAR, 2003.

PHỤ LỤC A
TÁI LẬP SỐ LIỆU

Mọi số trong báo cáo sinh ra từ các lệnh dưới đây, chạy từ trong thư mục dự án — mọi script phân giải ./images, ./output, ./test_images theo thư mục làm việc hiện tại. Hình vẽ sinh bằng make_figures.py. Bản đầy đủ của quy trình này, kèm giá trị kỳ vọng để đối chiếu sau mỗi bước và mục xử lý sự cố, nằm trong README.md của kho mã: <https://github.com/vo-hoang-kh4ng/comparing-character-glyph-images-Sino-N-m>

```
# Đặt PY=.venv/bin/python cho gọn
PY=.venv/bin/python

# chuẩn bị: Python 3.10+, một GPU ≥ 6GB
python -m venv .venv
$PY -m pip install -r requirements.txt
bash download_fonts.sh

# so sánh backend (mục V-A, V-B)
$PY search_all_chars_in_corpus.py \
--backend <backend> --device cuda
$PY benchmark_extractors.py
$PY analyze_metric_ceiling.py
$PY benchmark_search.py
```

```
# sinh dữ liệu huấn luyện, rồi fine-tune (mục III)
$PY render_fonts.py
$PY scan_augment.py --calibrate
$PY finetune_glyph.py --smoke
$PY finetune_glyph.py --epochs 6 --device cuda

# đánh giá trên scan thật (mục V-C)
$PY search_all_chars_in_corpus.py \
--backend chinese-clip-ft --device cuda
$PY evaluate_test_images.py \
--backend chinese-clip-ft --device cuda \
--labels output/label_sheets/labels.csv

# xếp hạng lại: bốn cấu hình (mục VII)
$PY -m pytest -q
S="--suite render --sample 600"
$PY evaluate_rerank.py $S --no-rerank
$PY evaluate_rerank.py $S --oracle-meta
$PY evaluate_rerank.py $S
$PY evaluate_rerank.py $S --normalize none

# hình vẽ trong bài
$PY make_figures.py
```

Tên đầy đủ của file nhãn là output/label_sheets/label_template.csv; ở trên viết tắt cho vừa cột.

Nhật ký huấn luyện đầy đủ, gồm cả lần ArcFace sập đổ ở mục VII-C, nằm trong output/finetune/train.log.

PHỤ LỤC B
MÔI TRƯỜNG VÀ CHI PHÍ TÍNH TOÁN

Bảng VI
MÔI TRƯỜNG ĐÃ KIỂM CHỨNG. CỘT VRAM LÀ MỨC ĐỈNH THỰC ĐO, KHÔNG PHẢI DUNG LƯỢNG CARD.

Thành phần	Giá trị
GPU (huấn luyện)	NVIDIA H100 80 GB, dùng đỉnh 3,7 GB
CPU	Intel Xeon Platinum 8462Y+, 16 nhân
RAM	188 GB
Python	3.10.12
PyTorch	2.6.0+cu124
Transformers	4.57.6
Chỉ mục	Faiss IndexFlatIP (chính xác)

Mức VRAM đỉnh 3,7 GB đạt được nhờ gradient_checkpointing cộng với việc xoá hẳn tháp văn bản sau khi nạp mô hình — bài toán chỉ dùng ảnh, nên text_model và text_projection không những không cần gradient mà còn không cần tồn tại. Hệ quả thực tế: **toàn bộ quá trình huấn luyện chạy vừa trên một GPU 6 GB phổ thông**, không cần phần cứng như trong Bảng VI.

Chi phí thời gian, đo trên cấu hình trên: render bảy font cho 26.044 ký tự khoảng 25 phút (chạy một lần); một epoch fine-tune ~4 phút, tổng 6 epoch ~25 phút; trích đặc trưng lại toàn bộ corpus ~3 phút; đánh giá 59 ảnh scan vài giây. Nghĩa là **toàn bộ chuỗi kết quả chính trong bài tái lập được trong khoảng một giờ**, chưa kể thời gian tải mô hình pretrain lần đầu.

Checkpoint fine-tune nặng 329 MB ở fp32. Bản phát hành trên Hugging Face lưu ở fp16 (165 MB); đây không phải xấp xỉ mà là không mất mát trên đường suy luận của chúng tôi, vì mã nạp mô hình gọi model.to(fp16) ngay sau khi nạp — chúng tôi đã kiểm bằng cách so toàn bộ 26.044 vector đặc

trưng của corpus giữa hai bản, sai khác tuyệt đối lớn nhất là 0,0.

PHỤ LỤC C
CẤU TRÚC GÓI NỘP

Corpus 26.044 ảnh và ba bảng `.xlsx` là dữ liệu đầu vào cho trước của đề bài nên không được đóng gói lại; `data/DATA.md` ghi rõ tên file và vị trí cần đặt để mọi script chạy được. Bộ kiểm thử chạy bằng `$PY -m pytest -q`; khi thiếu các bảng `.xlsx` nói trên sẽ có một số test bị bỏ qua — đó là hành vi đã định, không phải lỗi.

Bảng VII
NỘI DUNG `SUBMISSION.ZIP`.

Thư mục	Nội dung
<code>src/</code>	Toàn bộ mã nguồn nhóm viết, kèm <code>requirements.txt</code> và bộ kiểm thử.
<code>data/</code>	59 ảnh scan đánh giá, bảng nhãn, nhật ký huấn luyện, và <code>DATA.md</code> mô tả cách lấy lại các bảng <code>.xlsx</code> gốc.
<code>model/</code>	<code>MODEL.md</code> : đường dẫn tải trọng số và đoạn mã nạp. Trọng số không nằm trong gói, đúng theo yêu cầu — xem liên kết ở đầu bài.
<code>report/</code>	Báo cáo bản PDF và bản Doc, kèm <code>paper.tex</code> và thư mục <code>figs/</code> .